

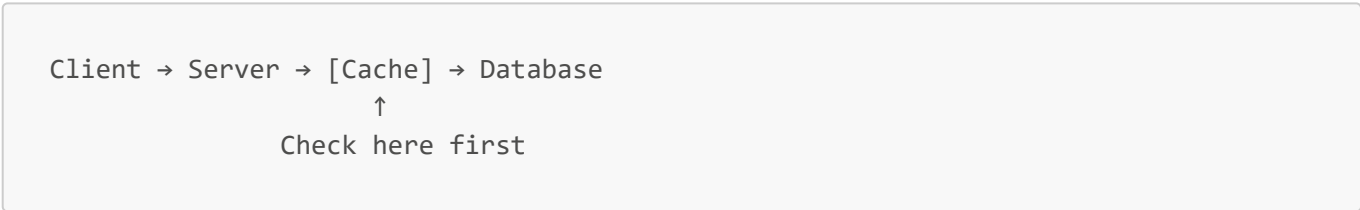
05a — Server Side Caching

Key Concepts

- **Caching** — storing a copy of data in a faster location to avoid re-fetching from the original source
 - **Cache hit** — requested data found in cache; returned immediately
 - **Cache miss** — data not in cache; fetched from DB, stored in cache, then returned
 - **TTL (Time To Live)** — how long a cache entry is valid before expiry
 - **Cache invalidation** — ensuring cache reflects current DB state when source data changes
 - **Cache eviction** — removing entries when cache is full to make room for new data
-

How It Works

1. Client sends request
2. Server checks cache first
3. **Hit** → return cached data (fast)
4. **Miss** → query DB → store in cache with TTL → return data
5. Next request for same data → cache hit



Types of Server Side Caching

Type	What's Cached	Tools
In-memory cache	Any data stored in RAM	Redis, Memcached
Database query cache	Results of expensive DB queries	MySQL query cache
Object cache	Serialized objects / API responses	Redis
Fragment cache	Part of a page or response	Custom logic

Eviction Strategies

Strategy	Removes
LRU — Least Recently Used	Entry untouched the longest
LFU — Least Frequently Used	Entry accessed fewest times
TTL-based	Entry closest to expiry
FIFO — First In First Out	Oldest inserted entry

Industry Examples

- **Twitter** — caches home timelines in Redis; avoids recomputing 800 tweets per load
 - **Amazon** — product detail pages cached; DB not hit on every view
 - **Stack Overflow** — runs lean infrastructure due to heavy Redis caching
-

Interview Questions

Q: What is server side caching?

Storing frequently requested data in a fast layer (usually in-memory) on the server so repeated requests skip the DB.

Q: What is a cache miss and what happens?

Data not found in cache — server queries DB, stores result in cache with TTL, returns it.

Q: What is TTL and what happens if it's too high or too low?

TTL = how long cached data is valid. Too high = stale data served. Too low = constant cache misses, no benefit.

Q: What is cache invalidation and why is it hard?

Ensuring cache is cleared/updated when DB data changes. Hard because TTL handles time-based expiry automatically, but event-driven invalidation (e.g., user updates profile mid-TTL) requires tracking what's cached per DB record and invalidating precisely.

Q: What is LRU eviction?

When cache is full, remove the entry that was least recently accessed to make room for new data.